

Model Context Protocol (MCP) for Ansible Automation Platform

Version 1, 2026-09-18

Welcome

Introduction

Course Title: Course title FIXME

Description: Course description FIXME

Duration: ...

Objectives

On completing this course, you should be able to:

- Observable, measurable objective...
- Observable, measurable objective...
- Observable, measurable objective...

Prerequisites

This course assumes that you have the following prior experience:

- Course or experience...
- Course or experience...
- Course or experience...



Use of Artificial Intelligence: This publication contains content or media that is generated or assisted by AI tools. All the Training deliverables have primarily human authorship, and all content was reviewed by a human domain expert.

Lab Environment

FIXME: Instructions for accessing the lab environment for this hands-on based training.

Chapter 1: Understanding MCP & AAP Integration

Welcome to the learning section of this course. In this chapter, you will understand what the Model Context Protocol is, how it works, and how Ansible Automation Platform uses it to connect with AI tools.

We'll break this down into bite-sized sections that follow a simple pattern: **What is it?** → **Why is it needed?** → **How does it work?**

What You'll Learn

By the end of this chapter, you will be able to:

- Define the Model Context Protocol and understand its role in the AI ecosystem
- Understand the architecture of MCP and how different components interact
- Explain Ansible Automation Platform's MCP server and its capabilities
- Describe the complete communication workflow from user to AI to AAP
- Identify and describe the 6 AAP MCP toolsets
- Understand the end-to-end workflow for deploying and using MCP

Time Required

This chapter takes approximately **30-45 minutes** to read and understand. Each section builds on the previous one, so go through them in order.

Ready? Start with Section 1 below!

Section 1: What is MCP?

What is the Model Context Protocol?

The **Model Context Protocol (MCP)** is an open-source standard that provides a universal way for AI models to interact with external data and tools.

Think of it as "**USB-C for the AI ecosystem**" — just like USB-C works with many different devices without custom adapters, MCP works with many different AI models and tools without requiring custom integrations.

[MCP as USB-C for AI] | *mcp-concept.png*

Instead of building a unique integration every time you want an AI model to interact with a new system, MCP provides a standardized method. This means:

- □ AI models can connect to different data sources without model-specific code

- □ You can swap AI providers without rewriting integrations
- □ Security and permissions are handled consistently

Why is MCP Needed?

Imagine you want to use ChatGPT to query your servers, while a colleague uses Claude to do the same thing. Without a standard protocol:

- ChatGPT integration team writes custom code to connect ChatGPT to your servers
- Claude integration team writes completely different code to do the same thing
- You have duplicated code, duplicated bugs, and duplicated maintenance

With MCP, both ChatGPT and Claude connect to the same MCP server. No duplication. No redundancy.

The Real Problem MCP Solves

AI models like Claude or ChatGPT are trained on data **frozen at a point in time**. They don't know:

- What jobs ran in your Ansible Automation Platform **yesterday**
- Which servers are currently online in your inventory
- What security events happened **today**
- Real-time metrics or current status

MCP solves this by giving AI models **real-time access** to your live systems. They can ask questions and get current answers, not guesses based on training data.

How Does It Work? (Simple Version)

MCP works in three simple steps:

MCP Flow

1. **You ask** → You send a natural language request to your AI tool
2. **AI translates** → The AI understands your intent and calls the appropriate MCP tool
3. **MCP delivers** → The MCP server fetches real data and returns it to the AI

Example

```
You: "List my recent Ansible jobs"
□
AI: "I need to call the job_list tool with recent=true"
□
MCP Server: Queries AAP and returns the list of jobs
□
AI: "Here are your last 5 jobs: ..."
```

Key Takeaways

- **MCP is a standard** for connecting AI models to external systems
- **It's like USB-C** — one standard that works with many AI tools
- **It solves a real problem** — giving AI real-time access to live data
- **It reduces duplication** — one integration instead of many
- **It's secure** — authentication and permissions are baked in

Next Steps

You understand **what** MCP is. Now let's understand **how** it's built — the architecture and components.

→ **Continue to Section 2: MCP Architecture**

Section 2: MCP Architecture

What are the Building Blocks?

MCP is built from **four main components**. Let's break them down:

Component 1: The AI Client (The Requester)

This is where **you** interact with the AI. It could be:

- **Claude Desktop** — A desktop application by Anthropic
- **Cursor IDE** — A code editor with AI built in
- **LibreChat** — A web-based chat interface
- **ChatGPT** — OpenAI's chatbot
- **Any MCP-compatible tool** you're using

Role: You enter natural language requests here. "Show me failed jobs from yesterday." The client sends your request to the AI model.

Component 2: The AI Model (The Translator)

This is the **large language model (LLM)** — the brain of the operation.

Examples: * Claude (Anthropic) * GPT-4 (OpenAI) * Qwen (Alibaba) * Any LLM that supports MCP

Role: It interprets your natural language request and figures out which MCP **tools** to call and with what parameters.

Example Interpretation

Your Input: "How many Windows servers do I have?"

AI Model Thinks: "The user wants server counts. I should:

1. Call `inventory_list` to get hosts
2. Filter for hosts with 'os_windows' group
3. Count them"

Component 3: MCP Server (The Gatekeeper)

The **MCP Server** is the bridge between the AI and your infrastructure.

Role: * Receives structured requests from the AI model * Validates the request using your API token
* Executes the request against your system (AAP, in this case) * Standardizes the response and sends it back to the AI

The MCP server **never trusts the AI directly**. It requires authentication and applies your system's security policies.

Component 4: Your System (The Executor)

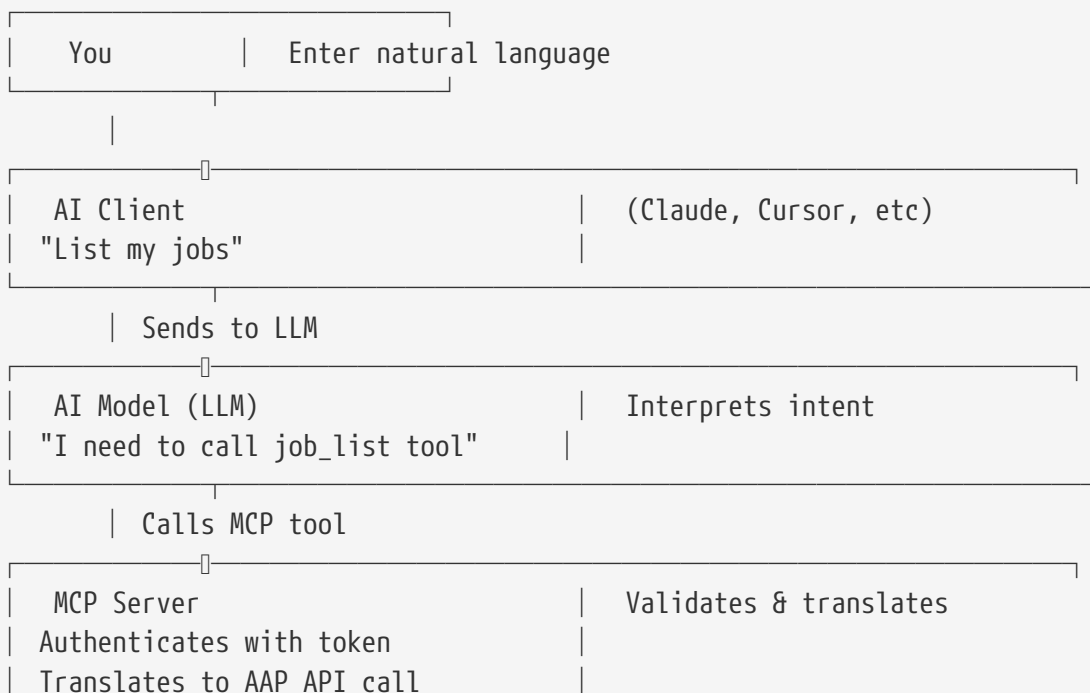
In this course, it's **Ansible Automation Platform (AAP)**, but it could be:

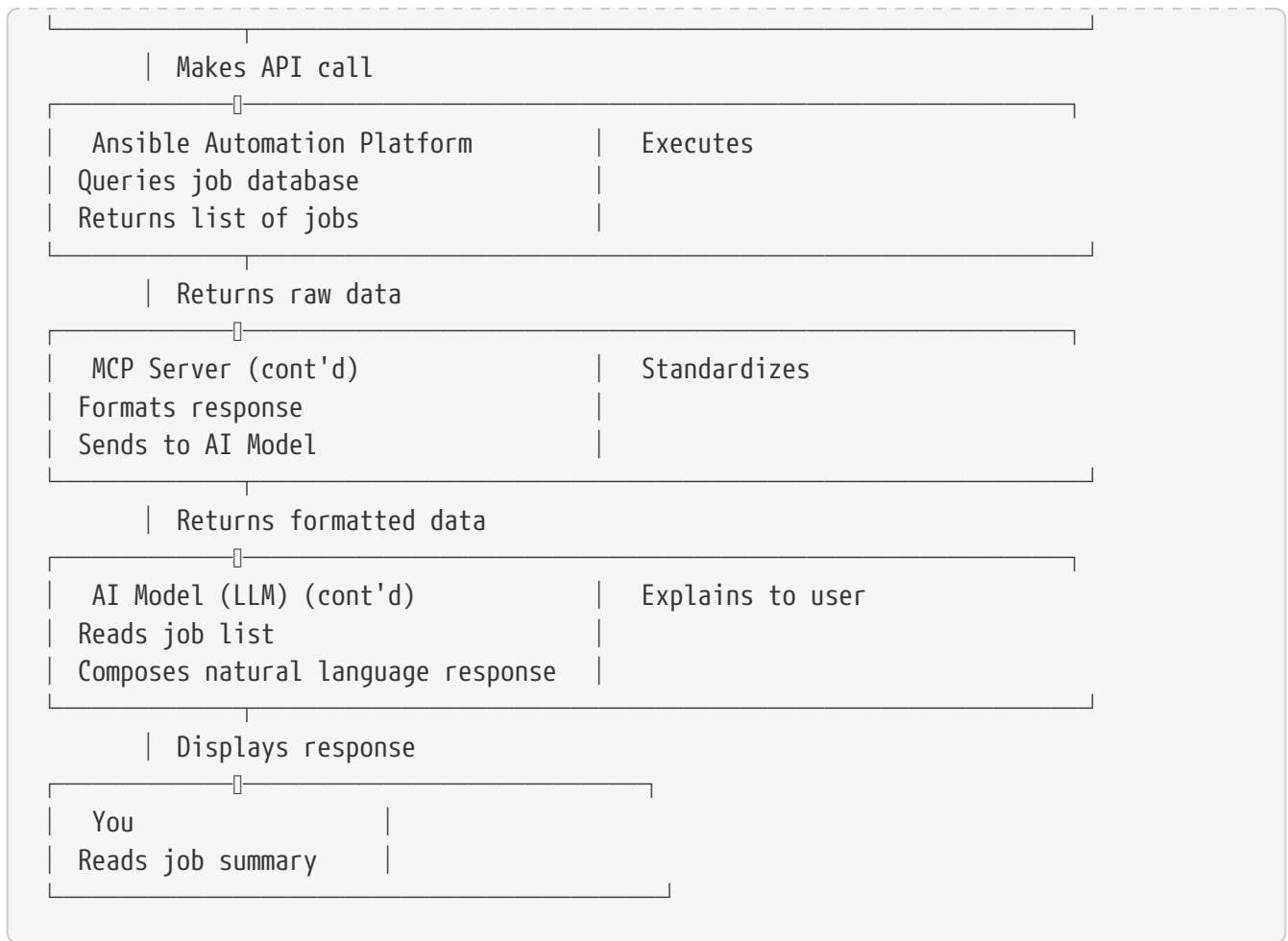
- Your cloud platform (AWS, Azure, GCP)
- Your internal database
- Your monitoring system
- Any system with an API

Role: Executes the actual action or returns the requested data.

How They Connect

Here's the complete architecture:





What About Security?

The MCP Server is the **security checkpoint**:

1. **Authentication**: Uses your AAP API token to verify requests
2. **Authorization**: Only allows actions your user account has permission to perform
3. **Standardization**: Ensures all requests follow the same format
4. **Validation**: Checks that parameters are valid before sending to AAP

Security Example

AI asks: "Delete all hosts"

MCP Server checks:

- ✓ Is the API token valid? YES
- ✓ Does the user have delete permission? NO
- ✗ DENIED - Permission rejected

Response to AI: "You don't have permission to delete hosts"

Key Takeaways

- **Four components** work together: Client, AI Model, MCP Server, Your System

- **MCP Server is the gatekeeper** — it validates and secures all requests
- **Tokens provide authentication** — proving who you are
- **RBAC is enforced** — you can only do what your user account allows
- **Response is standardized** — the AI gets consistent, predictable data

Next Steps

You understand the components. Now let's see how **Ansible Automation Platform** uses MCP.

→ **Continue to Section 3: AAP Integration & Toolsets**

Section 3: AAP Integration & Toolsets

What is the Ansible MCP Server?

The **Ansible MCP Server** is Ansible Automation Platform's implementation of the MCP standard.

It enables connecting AAP with any preferred external AI tool (such as Claude, Cursor, or ChatGPT). The AI tools can access information about your AAP environment and perform tasks using natural language.

Instead of logging into the AAP web UI manually, you can ask an AI: "Launch the 'Deploy Web Servers' job template" and it does it for you.

Why AAP Needs an MCP Server

AAP manages:

- * **Thousands of servers and devices**
- * **Complex automation workflows and job executions**
- * **Sensitive credentials and access control**
- * **User teams and role-based permissions**
- * **Audit logs and compliance data**

Before MCP, the only way to query this was:

1. Log into the web UI manually
2. Click through menus
3. Run API commands with **curl** or write custom scripts

MCP enables: Ask the AI in natural language, get instant answers.

The 6 AAP MCP Toolsets

The Ansible MCP Server exposes **6 separate toolsets**, each focused on a different area of AAP:

1. Job Management

What it does: Execute and monitor automation jobs.

Tools you can call:

- * List job templates
- * Start a job run
- * Check job status
- * View job output/logs
- * List recent jobs

Example queries:

```
"Show me failed jobs from the last 24 hours"  
"Launch the 'Configure Servers' job template"  
"What jobs are currently running?"
```

2. Inventory Management

What it does: Manage hosts, groups, and inventories.

Tools you can call: * List inventories * Get hosts in a group * List inventory groups * Get host details and variables * Filter hosts by properties

Example queries:

```
"Show me all Windows hosts"  
"Which hosts are in the 'production' group?"  
"List hosts that haven't checked in for 7 days"
```

3. System Monitoring

What it does: Monitor platform health and performance metrics.

Tools you can call: * Check AAP health status * Get execution node metrics * View job execution statistics * Monitor capacity and usage * Get system performance data

Example queries:

```
"What's the health status of my AAP instance?"  
"Show me job execution metrics for the past week"  
"What's the capacity utilization?"
```

4. User Management

What it does: Manage users, teams, and organizations.

Tools you can call: * List users and teams * Get team memberships * View organization details * Check user roles and permissions * Manage RBAC assignments

Example queries:

```
"List all users in my organization"  
"Who are the members of the ops team?"  
"What permissions does user 'alice' have?"
```

5. Security & Compliance

What it does: Access security policies, credentials, and audit logs.

Tools you can call: * View audit logs * List credentials * Check security settings * Review compliance data * Monitor access and changes

Example queries:

"Show me audit logs from the past 24 hours"
"What credentials are stored in AAP?"
"List all security-related activity"

6. Platform Configuration

What it does: Configure platform settings and resources.

Tools you can call: * List projects and their status * View execution environments * Get platform settings * Manage integrations * Configure system properties

Example queries:

"List all projects and their SCM status"
"What execution environments are available?"
"Show me platform configuration settings"

One Server, Many Endpoints

All 6 toolsets are served by **one MCP server** with **6 separate endpoints**:

Toolset	Endpoint Path
Job Management	/job_management/mcp
Inventory Management	/inventory_management/mcp
System Monitoring	/system_monitoring/mcp
User Management	/user_management/mcp
Security & Compliance	/security_compliance/mcp
Platform Configuration	/platform_configuration/mcp

When you configure an AI tool, you connect it to **all 6 endpoints** so the AI has access to everything.

Role-Based Access Control (RBAC)

The MCP server respects your AAP permissions. If you create an API token for a user with "read-only" permissions, the AI can only read data—it cannot launch jobs or make changes.

RBAC Example

Token created with SCOPE: Read

Allowed:

- ✓ List jobs
- ✓ View job output
- ✓ List inventories
- ✓ View hosts

Denied:

- ✗ Launch a job
- ✗ Create users
- ✗ Delete credentials
- ✗ Modify settings

This means: * **Admins** can use the AI to do anything they could do manually * **Read-only users** can only query information * **Team members** respect team-level permissions * **Audit logs** track all AI-driven actions

Key Takeaways

- **Ansible MCP Server** brings AI access to your AAP environment
- **6 toolsets** cover: jobs, inventory, monitoring, users, security, and configuration
- **1 server, 6 endpoints** — all can be connected at once
- **RBAC is enforced** — permissions from AAP flow through to MCP
- **Natural language** replaces manual web UI clicks and API calls

Next Steps

You understand the toolsets. Now let's see the **complete communication workflow**—how a request flows from user to AI to AAP.

→ **Continue to Section 4: Communication Workflow**

Section 4: Communication Workflow

The Full Journey: From User to AAP

Let's trace exactly what happens when you ask the AI a question. This is the **complete communication workflow**.

Step-by-Step Breakdown

Step 1: The User Initiates a Request

You: "Show me failed jobs from the past 7 days"

This is a natural language request sent to your AI tool (Claude, Cursor, ChatGPT, etc).

What's happening: * The AI client receives your message * It sends it to the AI model for interpretation

Step 2: The AI Model Interprets and Plans

```
AI Model (thinking):  
"The user wants:  
1. A list of jobs  
2. Filtered to only FAILED status  
3. From the last 7 days  
  
I need to call the job_list tool with:  
- status = "failed"  
- limit = 7 days  
"
```

What's happening: * The AI model understands your intent * It maps your request to available MCP tools * It prepares the structured request with parameters

Step 3: The MCP Server Receives the Request

```
Request from AI:  
{  
  "method": "tools/call",  
  "params": {  
    "name": "job_list",  
    "arguments": {  
      "status": "failed",  
      "limit_days": 7  
    }  
  }  
}
```

What's happening: * The MCP server receives a structured tool call * It validates the request format * It checks that the tool exists

Step 4: The MCP Server Authenticates

```
MCP Server checks:  
✓ API token is valid? YES  
✓ Token belongs to active user? YES  
✓ User has read permission? YES  
  
Status: AUTHENTICATED
```

What's happening: * The MCP server extracts your API token from the request headers * It verifies the token with AAP * It loads your user's permissions and RBAC settings

Step 5: The MCP Server Translates and Calls AAP

MCP Server translates to AAP API call:
GET /api/v2/jobs/?status=failed&created__gte=2024-09-08

Headers:

Authorization: Token <your-aap-api-token>
Content-Type: application/json

What's happening: * The MCP server converts the tool call into an AAP API request * It uses your API token to authenticate with AAP * It applies any RBAC restrictions from your user account

Step 6: AAP Executes the Query

AAP Database Query:
SELECT jobs WHERE status='failed' AND created > '2024-09-08'
RESULT: 3 failed jobs found

What's happening: * AAP queries its database * It applies your user's permission restrictions * It only returns jobs you're allowed to see * It returns the results as JSON

Step 7: The MCP Server Standardizes the Response

Raw AAP Response (200 JSON objects with 50 fields each):

```
{
  "count": 3,
  "results": [
    {
      "id": 456,
      "name": "Deploy Config",
      "status": "failed",
      "created": "2024-09-10T14:32:00Z",
      "started": "2024-09-10T14:32:05Z",
      "finished": "2024-09-10T14:45:00Z",
      ...50 more fields...
    },
    ...
  ]
}
```

MCP Server Standardizes:

```
{
  "jobs": [
    {
      "id": 456,
      "name": "Deploy Config",
      "status": "failed",
      "duration": "12 minutes",
```

```
    "failure_reason": "Host unreachable"
  },
  ...
]
}
```

What's happening: * The MCP server receives raw AAP data * It extracts only relevant fields * It formats data in a way the AI can easily understand * It simplifies complex structures

Step 8: The AI Model Processes the Data

AI Model receives standardized response
AI Model thinks:
"I have 3 failed jobs. Let me summarize them for the user
in a clear, readable format"

What's happening: * The AI receives the structured data * It reads and understands the job details
* It composes a human-readable summary

Step 9: The AI Client Displays the Response

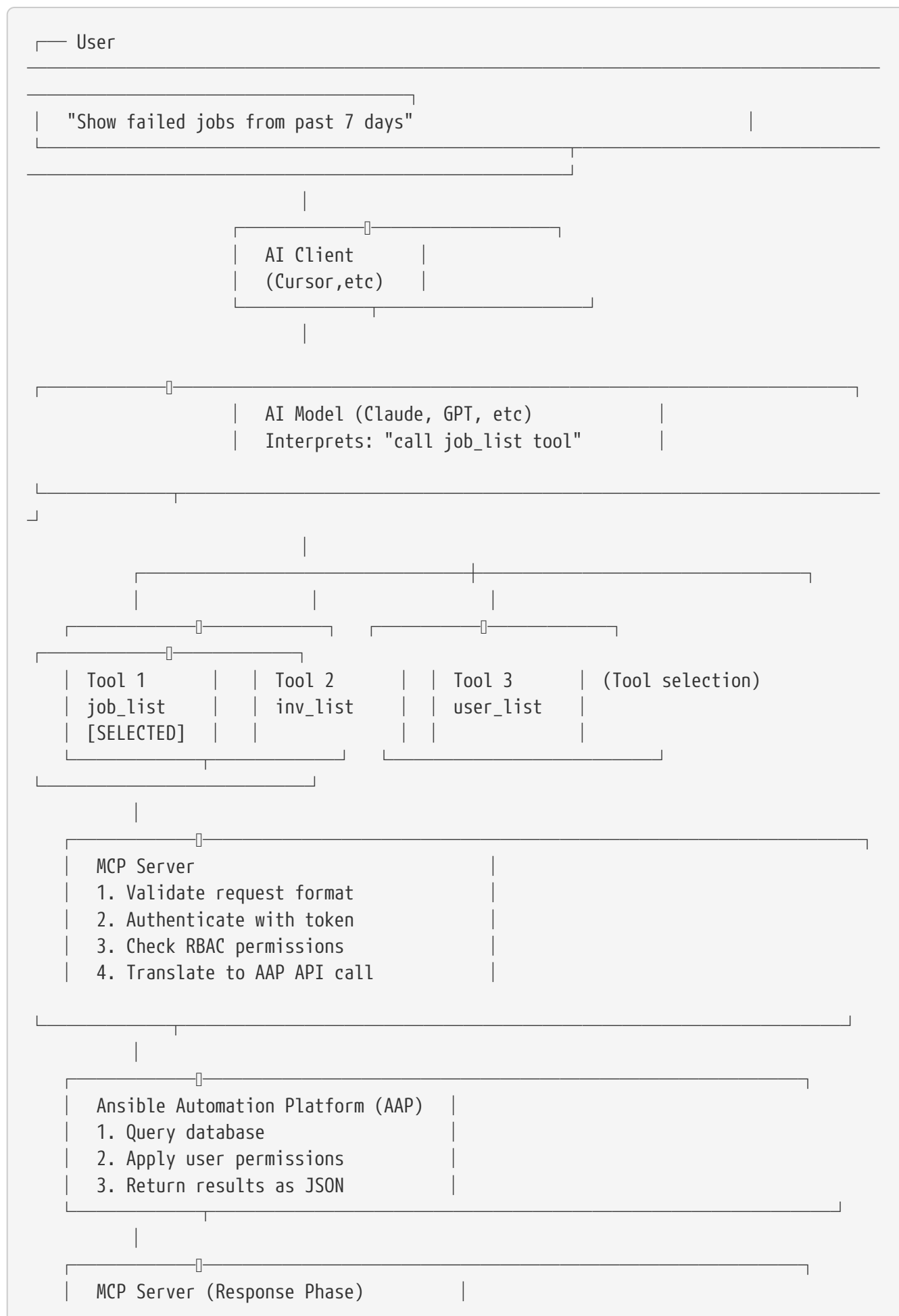
You see in your AI chat:

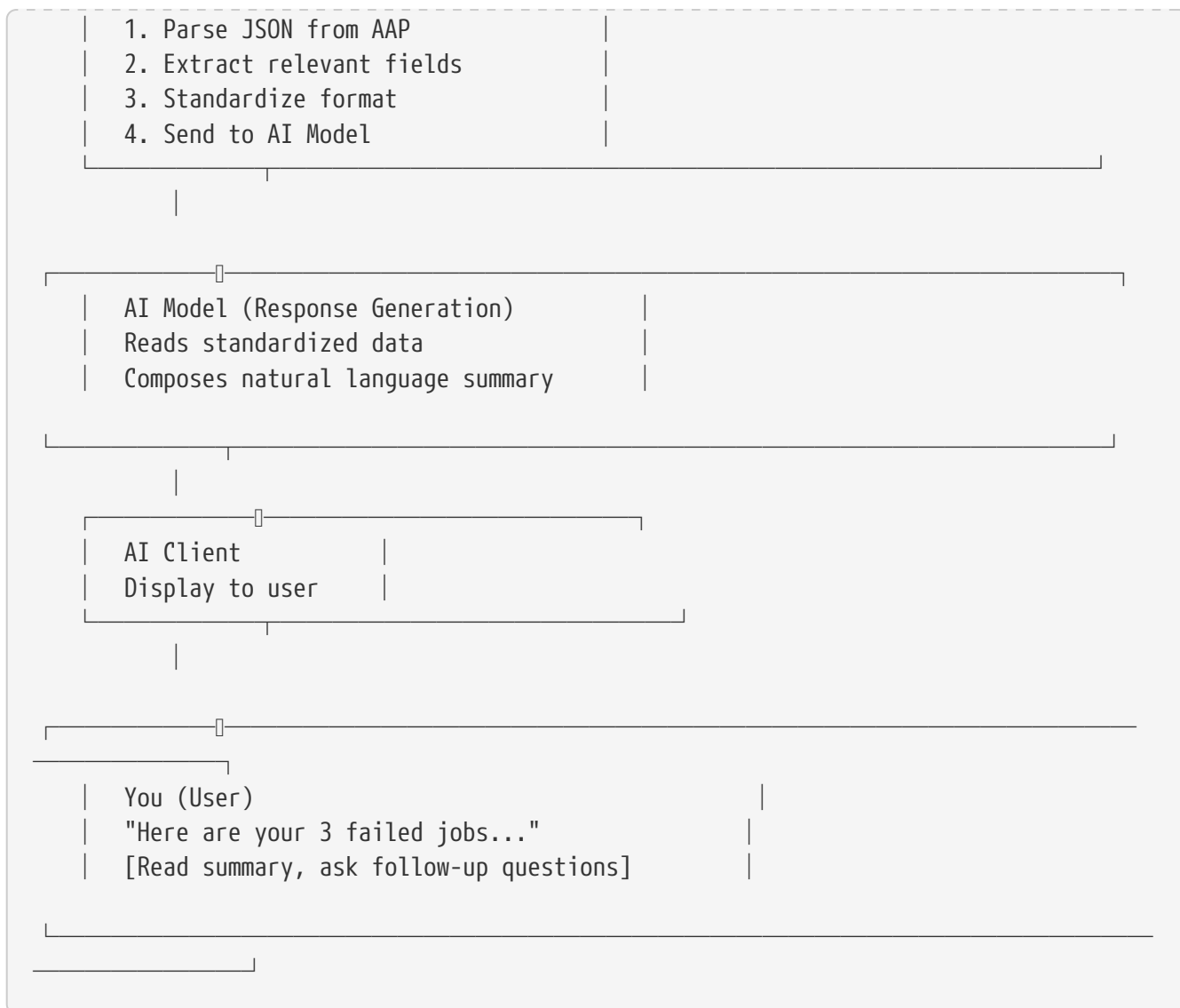
"You have 3 failed jobs from the past 7 days:

1. Deploy Config (Job #456)
 - Duration: 12 minutes
 - Status: FAILED
 - Reason: Host unreachable
 - Started: Sept 10, 2:32 PM
 2. Database Migration (Job #457)
 - Duration: 25 minutes
 - Status: FAILED
 - Reason: Permission denied
 - Started: Sept 11, 10:15 AM
 3. Cleanup Tasks (Job #458)
 - Duration: 5 minutes
 - Status: FAILED
 - Reason: Timeout
 - Started: Sept 12, 3:45 PM
- "

What's happening: * The AI client displays the formatted response * You read the summary * You can now ask follow-up questions

Visualized Workflow





Key Insights

Security Checkpoints

The workflow has **multiple security checkpoints**: 1. ✓ Token validation (is the request from a valid user?) 2. ✓ RBAC check (does the user have permission for this action?) 3. ✓ Data filtering (only show data the user is allowed to see)

Data Flow

Data flows in **two directions**: 1. **Request direction**: Natural language → Structured tool call → API call → Database query 2. **Response direction**: Database result → Formatted response → Natural language summary → Display

No Direct AI-to-AAP Access

Important: The AI model never talks directly to AAP.

The MCP server is always in the middle: * ✓ The AI cannot bypass authentication * ✓ The AI cannot exceed user permissions * ✓ All requests are logged and auditable * ✓ All data is standardized and validated

Key Takeaways

- **9 steps** happen in seconds when you ask a question
- **MCP Server is the gatekeeper** at every step
- **RBAC is enforced** — you can only see/do what your AAP user allows
- **Data is standardized** — complex AAP data is simplified for the AI
- **Security first** — tokens, permissions, and audit logs protect everything

Next Steps

You now understand the complete workflow. Let's look at what happens in real deployments.

→ **Continue to Section 5: End-to-End Workflow**

Section 5: End-to-End Workflow

Getting MCP Running: The Three Steps

Now that you understand **what** MCP is and **how** it works, here's what you actually **do** to make it work.

Every deployment follows the same three-step workflow:

Step	What Happens	Where
1. Deploy	Get the MCP server running on your infrastructure	Chapter 2, Lab 1
2. Authenticate	Create an API token so the AI tool can prove it's you	Chapter 2, Lab 1
3. Configure	Tell your AI tool where the MCP server is and provide the token	Chapter 2, Lab 2

Step 1: Deploy the MCP Server

Where Does It Run?

The Ansible MCP Server runs as a containerized service on your infrastructure. You can deploy it on:

- **OpenShift Container Platform (OCP)** — Most common for enterprise
- **RHEL with containers** — Simpler, on a single machine
- **Kubernetes** — Any Kubernetes cluster

How Do You Enable It?

For **OpenShift deployments**, you enable it in your `AnsibleAutomationPlatform` custom resource (a

Kubernetes YAML file).

```
apiVersion: aap.ansible.com/v1beta1
kind: AnsibleAutomationPlatform
metadata:
  name: aap
spec:
  mcp:
    disabled: false
    allow_write_operations: false
```

Set: * **disabled: false** → Enable the MCP server * **allow_write_operations: false** → Read-only (safe default for starting out)

The Kubernetes Operator will then: 1. Create a new Pod for the MCP server 2. Expose it with a network route 3. Handle certificates and networking

How Long Does It Take?

Usually **5-15 minutes** for the operator to reconcile and the MCP server to be ready.

Step 2: Generate an API Token

What's an API Token?

An **API token** is like a password, but: * ✓ Safer — it can't see your actual password * ✓ Restricted — you can give it only certain permissions * ✓ Temporary — you can delete it without changing your password * ✓ Auditable — you can see which token did what

How Do You Create One?

In Ansible Automation Platform:

1. Log in to the AAP web UI
2. Go to **Access Management** → **API Tokens**
3. Click **Create Token**
4. Enter:
 - **Name:** Something descriptive like **mcp-token** or **claude-integration**
 - **Scope:** Usually **Read** to start (safer), or **Read + Write** if you need to launch jobs
5. Click **Create** and copy the token immediately

❗ **Important:** The token is only displayed once. If you don't copy it now, you'll have to delete it and create a new one.

Example Token

```
a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6q7r8s9t0
```

This is what you'll provide to your AI tool later.

What Permissions Should It Have?

Scope	When to Use
Read Only	You want the AI to query information but not change anything. Great for learning and demos.
Read + Write	You want the AI to launch jobs, create users, etc. Use carefully with proper RBAC.

Recommendation: Start with **Read Only**. Upgrade to **Read + Write** once you're comfortable with the workflow.

Step 3: Configure Your AI Tool

What Are You Telling It?

You're giving your AI tool three pieces of information:

1. **Where is the MCP server?** (The URL)
2. **Who am I?** (Your API token)
3. **What can I access?** (The 6 toolsets)

How Do You Configure It?

This depends on which AI tool you're using:

Option A: Cursor IDE

```
# 1. Set environment variables
export AAP_SERVER="https://aap.example.com:8448"
export MY_SERVICE_TOKEN="your-api-token"

# 2. Open Cursor Settings → Tools & MCP
# 3. Paste the MCP configuration (includes all 6 toolsets)
# 4. Click Save and restart Cursor
```

Option B: Claude Desktop

```
# 1. Create a config file at:
# ~/.claude/claude.json

# 2. Add MCP server configuration with your AAP URL and token
```

3. Restart Claude

Option C: LibreChat (Web-Based)

```
# 1. Open LibreChat web interface
# 2. Go to Settings
# 3. Add MCP Server:
#   Name: aap
#   URL: https://aap.example.com:8448/mcp
#   Token: your-api-token
# 4. Click Connect
```

Example Configuration (JSON Format)

```
{
  "mcpServers": {
    "aap-job-mgmt": {
      "type": "http",
      "url": "https://aap.example.com:8448/job_management/mcp",
      "headers": {
        "Authorization": "Bearer YOUR_API_TOKEN"
      }
    },
    "aap-inventory": {
      "type": "http",
      "url": "https://aap.example.com:8448/inventory_management/mcp",
      "headers": {
        "Authorization": "Bearer YOUR_API_TOKEN"
      }
    },
    // ... (4 more toolsets)
  }
}
```

Verification: Is Everything Connected?

After configuration, test the connection:

```
# In your AI tool, type:
"What MCP tools are available?"

# The AI should respond:
"You have access to 6 AAP MCP toolsets with approximately
85 tools total:
- Job Management (15 tools)
- Inventory Management (12 tools)
```

- System Monitoring (18 tools)
- User Management (10 tools)
- Security & Compliance (14 tools)
- Platform Configuration (16 tools)"

If you see this, **congratulations!** Everything is working.

If not, check: * ✓ MCP server is running (check Kubernetes/OCP) * ✓ AAP URL is correct (no typos, correct port) * ✓ API token is valid (hasn't expired or been deleted) * ✓ Firewall allows connection to AAP

Example End-to-End Scenario

Here's what a real workflow looks like:

Monday 9:00 AM

You: "How many jobs failed this weekend?"

AI queries AAP → Returns 5 failed jobs

You: "Show me the details of job #456"

AI queries AAP → Returns error logs

You: "What was the failure?"

AI: "Host 'prod-web-02' was unreachable."

You: "Can I restart that host?"

AI: "Your token has read-only permission.
You'd need to create a new token with write access
or log into AAP directly to restart the host."

You: "Never mind. List all production hosts."

AI queries AAP → Returns 24 production hosts

You: "Are they all healthy?"

AI: "I'll check the monitoring data...
23 are healthy, but 'prod-db-03' has high CPU (87%)"

You: "Alert the team about the high CPU."

AI: "I've logged this. Here's the information to share:
Host: prod-db-03
CPU: 87%
Memory: 45%

Alert time: 9:15 AM"

This all happened **without logging into AAP directly** or writing any code.

Key Takeaways

- **3 simple steps:** Deploy → Authenticate → Configure
- **Deployment** takes 5-15 minutes on Kubernetes/OCP
- **Tokens** give controlled access to your infrastructure
- **Configuration** is just telling your AI tool where to find MCP
- **Verification** is asking the AI what tools it has access to

What's Next?

You now understand: * ✓ What MCP is * ✓ How the architecture works * ✓ What AAP provides * ✓
The communication flow * ✓ The deployment workflow

Chapter 2 is hands-on labs where you'll actually do these three steps yourself.

Ready? Let's get started with the labs!

→ **Continue to Chapter 2: Hands-On Labs**

Chapter 2: Hands-On Labs

Lab 1: Deploy MCP on OCP

Lab 2: Configure Your AI Tool

Lab 3: Run Queries & Troubleshoot

Appendix & Resources

MCP Cheat Sheet

Knowledge Check & Quiz

Troubleshooting Guide